

Klados: Branch-and-Price and SAT for Rooted Maximum Agreement Forests

Christian Inhetveen @ ORCID

Technische Universität Dortmund, Germany

Marcin Mennemann @ ORCID

Technische Universität Dortmund, Germany

Marius Maximilian Hille @ ORCID

Technische Universität Dortmund, Germany

Mart Hagedoorn @ ORCID

Technische Universität Dortmund, Germany

Kevin Buchin @ ORCID

Technische Universität Dortmund, Germany

Abstract

A maximum agreement forest cuts a set of rooted phylogenetic trees into the fewest pieces on which all trees agree. We describe *Klados* (κλάδος, Ancient Greek for “branch”), our exact solver for this problem in the PACE 2026 challenge. *Klados* shrinks the input with safe reduction rules, including a 3–2 rule for any number of trees, and then solves the rest by branch-and-price or SAT.

2012 ACM Subject Classification Theory of computation → Parameterized complexity and exact algorithms; Theory of computation → Integer programming; Applied computing → Bioinformatics

Keywords and phrases Rooted maximum agreement forest, branch-and-price, SAT

Digital Object Identifier 10.4230/LIPIcs.IPEC.2026.35

Category PACE Solver Description

Supplementary Material *Software (Source Code)*: <https://klados.buchinee.rs>

Acknowledgements Claude assisted with parts of the solver and text. The solver was extensively tested and peer-reviewed at PACE; the authors take full responsibility for all content.

1 Problem and Solver Overview

A *rooted binary phylogenetic tree* is a rooted tree whose leaves are labelled by *taxa* (for example, species) and whose internal vertices each have two children. We are given m such trees $\mathcal{T} = \{T_1, \dots, T_m\}$ on the same set X of n taxa, and want to cut them into the fewest pieces on which all trees agree (Figure 1).

Formally, for $Y \subseteq X$ let $T_i[Y]$ be the smallest subtree of T_i connecting the leaves in Y , and let $T_i|Y$ arise from it by *suppressing* vertices with one child (replacing them by a single edge). An *agreement forest* is a partition $\mathcal{F}$ of X into *components* such that (a) for every component Y , the trees $T_1|Y, \dots, T_m|Y$ have the same shape, and (b) within each tree T_i , the subtrees $T_i[Y]$ of different components share no vertex. $\text{OPT}(\mathcal{T})$ denotes the minimum number of components. The PACE 2026 challenge [7] allows any number of trees.

Klados works in three steps. First, it applies *reduction rules*, which shrink the instance while preserving optimal solutions, until none applies. A subtree that appears in all trees is replaced by one leaf [1]; a *chain* of taxa that hangs off a path in the same order in all trees is shortened to $m + 1$ taxa [5]; and the 3–2 rule of Section 2 deletes individual taxa. If all trees share a *cluster*, i.e., a set of taxa forming a complete subtree in every tree, the



© Christian Inhetveen, Marcin Mennemann, Marius Maximilian Hille, Mart Hagedoorn, and Kevin Buchin;

licensed under Creative Commons License CC-BY 4.0

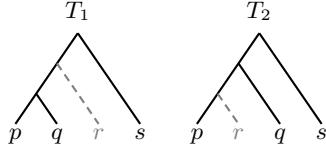
21st International Symposium on Parameterized and Exact Computation (IPEC 2026).

Editors: Tuukka Korhonen and Michael Lampis; Article No. 35; pp. 35:1–35:4



Leibniz International Proceedings in Informatics

Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany



■ **Figure 1** T_1 has the cherry $\{p, q\}$ and T_2 the subtree $((p, r), q)$, so (p, q, r) is a 3–2 configuration. Cutting the dashed edges gives the optimal agreement forest $\{\{p, q, s\}, \{r\}\}$; on $\{p, q, s\}$, both trees have the shape $((p, q), s)$.

instance is split there into smaller instances whose optimal forests combine into an optimal forest of the input [9, 5]. Second, Klados solves these by branch-and-price (Section 3), or by SAT (Section 4) if the original input has $m \geq 3$ trees and $n \leq 110$ taxa, where the LP bound tends to be weak and the SAT encoding stays small. Third, undoing the reductions gives an optimal forest for the input. Klados placed third in the exact track, solving 866 of 1200 runs (400 instances, three runs each).

2 The Rooted 3–2 Rule for Many Trees

Two leaves form a *cherry* if they have the same parent. The 3–2 rule [4] was designed for two trees; we extend it to any number of trees. Distinct taxa p, q, r form a *3–2 configuration* if (i) every tree contains either the cherry $\{p, q\}$ or the subtree $((p, r), q)$, i.e., the cherry $\{p, r\}$ with sibling q , and (ii) both cases occur (Figure 1). Let $\mathcal{T}'$ arise from $\mathcal{T}$ by deleting r from every tree and suppressing vertices with one child. Removing r costs exactly one component:

► **Lemma 1.** *If (p, q, r) is a 3–2 configuration in $\mathcal{T}$, then $\text{OPT}(\mathcal{T}) = 1 + \text{OPT}(\mathcal{T}')$.*

Proof sketch. Adding $\{r\}$ to a forest of $\mathcal{T}'$ gives one of $\mathcal{T}$. Conversely, we show that some optimal forest of $\mathcal{T}$ contains $\{r\}$, i.e., r is *alone*; removing it gives a forest of $\mathcal{T}'$. Trees with the cherry $\{p, q\}$ and trees with $((p, r), q)$ disagree on the shape of $\{p, q, r\}$, so these three taxa never share a component. If two of them share a component Y , the third must be alone, as otherwise two components overlap in some tree. If $r \in Y$, then r and its partner in Y form a cherry of $T_i|Y$ in trees with $((p, r), q)$, hence, by compatibility, in all trees, so replacing r by the lone taxon only renames a leaf and keeps the forest valid; thus we may assume $\{p, q\} \subseteq Y$ and r alone. If p, q, r lie in three components and r is not alone, then in a tree with $((p, r), q)$ the component of r cuts off p and q , so both are alone. Merging them saves a component, which pays for separating r . ◀

After deleting r , $\{p, q\}$ is a cherry in every tree and is merged into one leaf by the subtree rule, possibly creating new 3–2 configurations. With three or more trees there are often several candidates for r , and the choice affects how far this cascade goes. Klados runs the reductions three times, picking the first candidate, the last, or the one expected to create the most new common cherries, and keeps the smallest result.

3 Branch-and-Price

Integer program. Call $Y \subseteq X$ *compatible* if all trees $T_i|Y$ have the same shape; $\mathcal{C}$ is the set of compatible sets. Let $V_i[Y]$ be the non-leaf vertices of $T_i[Y]$, including those suppressed in $T_i|Y$. We use a 0/1 variable x_Y for each $Y \in \mathcal{C}$ and minimize $\sum_{Y \in \mathcal{C}} x_Y$ subject to

$$\sum_{Y \ni t} x_Y = 1 \quad (t \in X), \quad (\text{I})$$

$$\sum_{Y: v \in V_i[Y]} x_Y \leq 1 \quad (i \in \{1, \dots, m\}, v \text{ internal in } T_i). \quad (\text{II})$$

Constraint (I) makes the chosen sets partition X ; (II) puts each internal vertex of T_i in at most one subtree $T_i[Y]$ of a chosen set Y , so these subtrees are vertex-disjoint. Thus the solutions are exactly the agreement forests, extending [6, 3] from two to any number of trees.

Since $\mathcal{C}$ is huge, Klados uses branch-and-price: it solves each linear relaxation ($0 \leq x_Y \leq 1$) by column generation, i.e., over a small subset of the variables, adding a variable only when the dual solution shows that it can lower the objective. This *restricted master* starts with single taxa, sets found by heuristics, and no constraints (II); violated ones are added before each pricing round. The 2-approximation of Chen et al. [2] gives a good start for two trees and, for any two of the trees, a lower bound on $\text{OPT}(\mathcal{T})$.

Pricing: finding better components. LP duality gives each constraint a price: α_t for taxon t in (I), and the negated dual $\beta_{i,v} \geq 0$ for vertex v of T_i in (II), which is zero while the constraint is omitted. As the column of x_Y has ones in the rows of the taxa in Y and of the vertices in each $V_i[Y]$, its *reduced cost* is $1 - \text{score}(Y)$, where

$$\text{score}(Y) = \sum_{t \in Y} \alpha_t - \sum_{i=1}^m \sum_{v \in V_i[Y]} \beta_{i,v}. \quad (1)$$

Pricing means finding a compatible set with score above one or proving that none exists.

Klados prices by dynamic programming over pairs of taxa. The two children of the root of $T_i[Y]$ split a compatible set Y with $|Y| \geq 2$ into two *sides*, the same in all trees as all $T_i|Y$ agree. For p, q on opposite sides, the root of $T_i[Y]$ is $r_i(p, q) = \text{lca}_{T_i}(p, q)$, and the side of p lies in $A(p | q) = \bigcap_i L_i(c_i(p | q))$, where $c_i(p | q)$ is the child of $r_i(p, q)$ above p and $L_i(v)$ are the leaves below v . Conversely, compatible sets within $A(p | q)$ and $A(q | p)$ join to a compatible one. So the best score $P(p, q)$ of a compatible set with p, q on opposite sides is

$$P(p, q) = S(p | q) + S(q | p) - \sum_i \beta_{i, r_i(p, q)}, \quad S(p | q) = \max_{w \in A(p | q)} (P(p, w) - \pi_{p|q}(w)), \quad (2)$$

where $S(p | q)$ is the best contribution of the side of p , $\pi_{p|q}(w)$ is the price of the vertices strictly between $r_i(p, q)$ and $r_i(p, w)$, summed over all trees, and $w = p$ stands for the side $\{p\}$, with $P(p, p) = \alpha_p$ and $r_i(p, p) = p$ (Figure 2). For $w \neq p$, the best set for (p, w) indeed lies in $A(p | q)$, as $r_i(p, w)$ is below $c_i(p | q)$ in all trees. Each step moves down in all trees, so the recursion ends. Pricing maximizes $P(p, q)$ over all pairs; with prefix sums of β along root paths, each term costs $O(1)$. Two trees use a faster dynamic program over vertex pairs.

Branching and proving optimality. If the LP solution is fractional, some taxa p, q have *togetherness* $z_{pq} = \sum_{Y \supseteq \{p, q\}} x_Y$ strictly between 0 and 1 [8]; Klados picks p, q with z_{pq} closest to $1/2$ and creates two subproblems: p and q are in the same component, or they are not. Sets that break a subproblem's decisions are switched off there. To find good forests early, Klados also picks disjoint sets from the LP solution and adds the other taxa as singletons.

A subproblem is discarded if its LP value, a lower bound, cannot beat the best forest found. The bound is valid once a complete pricing run, which may ignore the decisions, finds

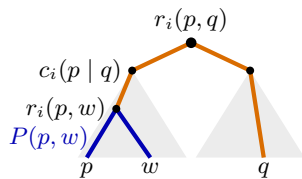


Figure 2 Pricing in one tree T_i . Here the side of q is q alone, and the side of p is a smaller set with p and w on opposite sides of $r_i(p, w)$, whose best score $P(p, w)$ is already known (blue). The recursion subtracts the prices of the root and the connecting paths (orange). Grey: leaves below the two children of the root.

no improving set; omitting constraints (II) only lowers it. Branching covers every grouping of the taxa, and each accepted solution satisfies (I)–(II), so the search returns an optimal forest.

4 SAT Encoding

We encode “is there an agreement forest with at most k components?” as a Boolean formula for the SAT solver CaDiCaL, describing a forest by the edges it cuts. Variable $d_{i,e}$ is true if edge e of T_i is cut, and c_{pq} , shared by all trees, is true if p, q stay connected. We require the left condition for every tree T_i and pair p, q of taxa, and the right one once:

$$c_{pq} \iff \bigwedge_{e \in \text{path}_{T_i}(p,q)} \neg d_{i,e}, \quad \sum_{e \in E(T_1)} d_{1,e} \leq k - 1. \quad (3)$$

Thus p, q are connected exactly if no edge on their path is cut; as c_{pq} is shared, all trees split the taxa alike, and at most $k - 1$ cuts in T_1 leave at most k components.

For agreement, note that a tree on three taxa is determined by its cherry. If two trees give p, q, r different shapes, the clauses $(\neg c_{pq} \vee \neg c_{pr})$, $(\neg c_{pq} \vee \neg c_{qr})$, $(\neg c_{pr} \vee \neg c_{qr})$ allow at most one connected pair, which by transitivity means that p, q, r do not share a component.

The formula thus describes agreement forests exactly: cut edges yield disjoint pieces, two trees have the same shape if they agree on every three taxa, and every agreement forest with at most k components arises from at most $k - 1$ cuts per tree. Starting from a heuristic forest, Klados lowers k , reusing learned clauses and the previous solution, until the formula is unsatisfiable or k hits a lower bound from pairs of trees; the last forest found is then optimal.

References

- 1 Magnus Bordewich and Charles Semple. On the computational complexity of the rooted subtree prune and regraft distance. *Annals of Combinatorics*, 8(4):409–423, 2005. doi:10.1007/s00026-004-0229-z.
- 2 Zhi-Zhong Chen, Youta Harada, Yuna Nakamura, and Lusheng Wang. Faster exact computation of rSPR distance via better approximation. *IEEE/ACM Transactions on Computational Biology and Bioinformatics*, 17(3):916–929, 2020. doi:10.1109/TCBB.2018.2878731.
- 3 Martin Frohn, Steven Kelk, and Simona Vychytilova. A branch-&-price approach to the unrooted maximum agreement forest problem. *Operations Research Letters*, 63:107364, 2025. doi:10.1016/j.orl.2025.107364.
- 4 Steven Kelk, Simone Linz, and Ruben Meuwese. Cyclic generators and an improved linear kernel for the rooted subtree prune and regraft distance. *Information Processing Letters*, 180:106336, 2023. doi:10.1016/j.ipl.2022.106336.
- 5 Steven Kelk, Ruben Meuwese, and Leo van Iersel. A kernel for the maximum agreement forest problem on multiple binary phylogenetic trees. arXiv:2602.22281, 2026. doi:10.48550/arXiv.2602.22281.
- 6 Neil Olver, Frans Schalekamp, Suzanne van der Ster, Leen Stougie, and Anke van Zuylen. A duality based 2-approximation algorithm for maximum agreement forest. *Mathematical Programming*, 198(1):811–853, 2023. doi:10.1007/s10107-022-01790-y.
- 7 PACE Challenge. PACE 2026 – (rooted) maximum-agreement forest, 2026. URL: <https://pacechallenge.org/2026/maf/>.
- 8 David M. Ryan and Brian A. Foster. An integer programming approach to scheduling. In Anthony Wren, editor, *Computer Scheduling of Public Transport: Urban Passenger Vehicle and Crew Scheduling*, pages 269–280. North-Holland, Amsterdam, 1981.
- 9 Chris Whidden, Robert G. Beiko, and Norbert Zeh. Fixed-parameter algorithms for maximum agreement forests. *SIAM Journal on Computing*, 42(4):1431–1466, 2013. doi:10.1137/110845045.